

Modelagem do Problema do Caixeiro Viajante: Têmpera Simulada e Algoritmo Genético

Felipe dos Reis Garcez¹, Gabriel Affonso Borges Caballero¹

¹Universidade Tecnológica Federal do Paraná

`felipegarcez@alunos.utfpr.edu.br, gabrielaffonso@alunos.utfpr.edu.br`

Abstract. *This paper presents a shared computational model of the Euclidean Traveling Salesperson Problem for simulated annealing and a genetic algorithm. Tours are represented as permutations and evaluated with a distance matrix. In ten instances with 40 cities, both methods improved on the nearest-neighbor greedy heuristic, with simulated annealing achieving the lower median relative cost.*

Resumo. *Este trabalho apresenta uma modelagem computacional compartilhada do Problema do Caixeiro Viajante euclidiano para Têmpera Simulada e Algoritmo Genético. As rotas são permutações avaliadas por uma matriz de distâncias. Em dez instâncias de 40 cidades, ambos superaram a heurística gulosa de vizinho mais próximo, e a Têmpera Simulada obteve menor custo mediano relativo.*

1. Introdução

O Problema do Caixeiro Viajante consiste em encontrar um percurso de menor custo que visite todas as cidades e retorne à origem. A enumeração das ordens de visita cresce fatorialmente com o número de cidades, motivando o estudo de métodos que exploram parte do espaço de soluções [Russell and Norvig 2022].

Neste trabalho, implementamos Têmpera Simulada (TS) e Algoritmo Genético (AG) sobre uma mesma modelagem. Descrevemos as escolhas de representação, os operadores e sua relação com a viabilidade e a avaliação das soluções. A TS modifica uma rota atual e pode aceitar piores temporárias, enquanto o AG mantém uma população e combina indivíduos por reprodução.

O código-fonte está disponível no repositório do projeto¹.

2. Definição do problema e modelagem

Adotamos um grafo completo, não direcionado e ponderado $G = (V, E)$, com $V = \{0, \dots, n-1\}$, $n \geq 3$, e uma coordenada $p_i = (x_i, y_i) \in \mathbb{R}^2$ para cada cidade. O peso de uma aresta sendo então a distância euclidiana:

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \quad (1)$$

Definimos uma solução como uma sequência ordenada de visitas, representada por uma permutação $\pi = (\pi_0, \pi_1, \dots, \pi_{n-1})$ do conjunto de cidades V . Buscamos encontrar a permutação que minimize o custo total da rota, definido pela soma das distâncias entre as cidades consecutivas, incluindo o retorno da última cidade para a cidade de origem:

$$f(\pi) = d(\pi_0, \pi_1) + d(\pi_1, \pi_2) + \dots + d(\pi_{n-1}, \pi_0). \quad (2)$$

Onde $d(a, b)$ é a distância euclidiana entre a cidade a e a cidade b . No código, a rota é representada como uma lista de índices, tal qual $[0, 2, 1, 3]$. Cada cidade aparece exatamente uma

¹<https://github.com/FelipeReis96/Sistemas-Inteligentes.git>

vez e o retorno à primeira cidade para fechar o ciclo é calculado implicitamente pela função de avaliação.

Embora fosse possível gerar diretamente uma matriz de custos entre pares de cidades, optamos por gerar coordenadas no plano e derivar as distâncias a partir delas. Essa escolha faz com que as instâncias tenham consistência geométrica entre as cidades.

Escolhemos o grafo completo para simplificar a modelagem dos algoritmos. Como existe uma conexão entre qualquer par de cidades, toda permutação de cidades é uma solução viável do problema. Assim não precisamos fazer verificações de conexões inexistentes ou impossíveis, que acrescentariam complexidade à implementação.

As distâncias são calculadas previamente e armazenadas em uma matriz de espaço $O(n^2)$. A escolha por essa estrutura justifica-se pela velocidade de execução: a consulta do custo de uma aresta é feita em tempo $O(1)$, o que torna a avaliação completa de uma rota dependente apenas de $O(n)$. Se utilizássemos uma lista de adjacências, demoraria mais e não teria melhoria de espaço por conta do grafo completo. Em instâncias maiores, nas quais as metaheurísticas avaliam milhares de rotas por segundo, esse custo extra degradaria severamente o desempenho.

3. Metodologia

3.1. Implementação e reprodutibilidade

A implementação foi feita em Python, organizada a partir de um módulo `problema.py` que concentra a parte de gerar cidades, construir a matriz e avaliar rotas. Os dois algoritmos usam desse módulo para avaliar suas soluções pelo mesmo critério, definido na Equação 2. Na parte de geração aleatória, adotamos sementes configuráveis para controlar os sorteios. Desse jeito, podemos repetir uma mesma configuração em mais de uma execução, o que facilita para comparar de forma justa mudanças de implementação.

3.2. Têmpera Simulada

A TS recebe uma permutação como rota inicial. A cada iteração, uma rota candidata é produzida pelo operador de vizinhança. Como esse operador apenas reorganiza as cidades, a permutação e a viabilidade da rota são preservadas. Calculamos $\Delta = f(\pi') - f(\pi)$, onde π' é a rota candidata. Se $\Delta \leq 0$, a rota é aceita. Caso contrário, é aceita com probabilidade $\exp(-\Delta/T)$ para $T > 0$.

A temperatura segue o resfriamento geométrico $T \leftarrow \alpha T$, com $0 < \alpha < 1$. A execução termina após I iterações. Guardamos a melhor rota separadamente da atual, pois a aceitação de pioras também pode afastar a busca de uma solução melhor já visitada. Se a temperatura numérica chegar a zero, pioras são rejeitadas e as iterações restantes continuam.

3.3. Decisões de modelagem na Têmpera Simulada

Para a geração da solução inicial, a rota inicial é construída pela heurística do vizinho mais próximo. Partimos da cidade 0 e escolhemos repetidamente a cidade ainda não visitada mais próxima, com desempate pelo menor índice. Dessa forma, a TS começa com uma rota estruturada e válida.

Já para modelagem na geração de vizinhos, usamos a inversão de intervalo: sorteamos duas posições e invertemos toda a sequência entre elas. Por exemplo, $[0, 1, 2, 5, 3, 4]$ pode se tornar $[0, 3, 5, 2, 1, 4]$. Como as distâncias são simétricas, esse movimento mantém as ligações internas do trecho e substitui as ligações em suas extremidades, podendo desfazer cruzamentos sem repetir ou remover cidades.

Já sobre a função de aceitação `deve_aceitar`, escolhemos a implementação clássica já descrita para esse algoritmo. A lógica é se distanciar do hill climbing, aceitando também, às vezes, caminhos inicialmente piores.

Optamos por usar uma quantidade fixa de iterações como única condição de parada da TS para tornar o custo computacional mais previsível e facilitar a comparação justa com o AG. Poderíamos ter usado a também válida condição $T \leq T_{\text{FINAL}}$, mas poderia fazer com que o programa se encerrasse cedo demais e prejudicar a comparação.

Por fim, calculamos o α para que a temperatura, partindo de 100, chegasse a 0,1 no final das iterações dos experimentos realizados. Chegamos a $\alpha \approx 0,9986$.

3.4. Algoritmo Genético

O AG mantém uma população de soluções que evolui ao longo de gerações. No caixeiro viajante, cada indivíduo representa uma rota completa. Seu cromossomo é a sequência de cidades na ordem de visita, e o custo corresponde à distância total percorrida.

A cada geração, avaliamos as rotas e atribuímos uma aptidão (*fitness*) a cada indivíduo. Essa aptidão orienta a seleção dos pais, dando mais chances às melhores rotas. Os pais selecionados podem combinar suas sequências por cruzamento, formando filhos. A mutação introduz alterações nesses filhos e permite explorar outras ordens de visita.

Depois de avaliar os filhos, definimos quais indivíduos compõem a próxima população. Esse ciclo de seleção, reprodução e sobrevivência se repete até a condição de parada. Ao longo da execução, registramos o melhor custo e o número acumulado de avaliações para acompanhar a evolução da busca.

3.5. Decisões de modelagem no Algoritmo Genético

Começamos com 50 rotas aleatórias para oferecer diferentes pontos de partida à busca. Cada rota é uma permutação, e os operadores precisam preservar essa estrutura para que nenhuma cidade seja repetida ou esquecida.

Para calcular o fitness, ordenamos os indivíduos pelo custo e distribuímos pesos de 1,5 ao melhor até 0,5 ao pior. Empates recebem o peso da posição média do grupo. A seleção por roleta sorteia os pais com probabilidade proporcional a esses pesos. Assim, a chance de seleção depende da posição da rota no ranking, e não do tamanho numérico da diferença de custo entre as rotas.

No cruzamento, usamos o PMX (*Partially Mapped Crossover*) para combinar rotas sem repetir ou excluir cidades. Ele copia um trecho de um pai e usa o mapeamento entre os trechos dos dois pais para resolver repetições nas demais posições. Essa lógica dispensa a necessidade de criar funções extras e lentas só para "consertar" rotas quebradas. O cruzamento ocorre com probabilidade 0,8 por par, e quando não ocorre, faz os filhos serem cópias dos pais.

Além disso, os pontos de corte desse trecho copiado são sorteados aleatoriamente. Se nós fixássemos o tamanho do corte (por exemplo, sempre pegar exatamente a metade), o algoritmo ficaria engessado. Ao sortear os índices de início e fim, permitimos que a evolução crie todo tipo de descendente: às vezes muito parecidos com um dos pais (o que é bom se o pai for muito apto), e às vezes misturas radicais (o que é ótimo para descobrir rotas totalmente novas).

Na mutação, percorremos cada posição do filho e, com probabilidade 0,05, invertemos o trecho até outra posição sorteada. A inversão preserva a permutação e pode desfazer cruzamentos geométricos. Um filho pode sofrer várias inversões, sendo que a inversão da rota inteira é evitada por apenas mudar o sentido do mesmo ciclo.

Para formar a próxima geração, mantemos os 50 melhores entre pais e filhos, permitindo indivíduos repetidos. Essa sobrevivência elitista preserva o melhor custo encontrado, mas reduz a diversidade.

4. Resultados e discussão

Para avaliar a qualidade das rotas, decidimos fazer a comparação entre as rotas geradas e a solução ótima para cada instância. Usamos o algoritmo exato de Held-Karp a fim de encontrar o custo ótimo de cada conjunto de cidades. Assim, pudemos medir quanto cada solução encontrada se distancia do melhor resultado possível.

Em seguida, executamos uma bateria de testes com cinco instâncias de 18 cidades e dez repetições por algoritmo em cada instância, totalizando 50 execuções para a TS e 50 para o AG. Os dois métodos receberam o mesmo orçamento de 5.050 avaliações da função objetivo por execução. Comparamos os resultados pelo desvio percentual em relação ao ótimo: valores menores indicam soluções melhores e zero indica que o ótimo foi alcançado.

4.1. Configurações iniciais

A Figura A.1, no Apêndice A, apresenta os desvios obtidos pelas configurações iniciais dos algoritmos. Nessa bateria inicial, os resultados dos dois métodos se sobrepõem bastante. O AG foi ligeiramente melhor, obtendo desvio mediano de 6,06%, menor que os 7,70% da TS, e alcançando o ótimo em 26% das execuções, contra 24% da TS. Enquanto isso, a TS apresentou duas execuções atípicas, com desvios acima de 35%, que aparecem como círculos acima da caixa.

4.2. Evolução da Têmpera Simulada

A configuração inicial da TS, avaliada na Seção 4.1, combinava uma rota inicial aleatória com a vizinhança por troca simples de duas cidades. A partir dela, revisamos primeiro a inicialização e depois a vizinhança, alterando apenas um componente por bateria. Mantivemos as mesmas instâncias, sementes da busca, temperaturas e orçamento de 5.050 avaliações.

A primeira decisão que revimos na TS foi a construção da rota inicial. Comparamos a rota aleatória da primeira versão com a heurística do vizinho mais próximo, que começa na cidade 0 e escolhe repetidamente a cidade não visitada mais próxima. Como podemos ver na Figura A.2, no Apêndice A, o vizinho mais próximo reduziu o desvio mediano inicial de 150,78% para 21,37%. Considerando a busca inteira, a diferença foi menor, mas permaneceu favorável: o desvio mediano caiu de 7,70% para 6,12%, e a taxa de sucesso aumentou de 24% para 30%. O resultado atendeu o critério definido antes da execução, por isso adotamos o vizinho mais próximo como padrão.

Com a nova inicialização fixada, comparamos a simples troca de duas cidades com a inversão de intervalo. Nossa hipótese era que a mudança das ligações nas extremidades do trecho facilitaria a remoção de cruzamentos e produziria rotas menores. Como mostra a Figura A.3, no apêndice, a inversão reduziu o desvio mediano de 6,12% para zero e elevou a taxa de sucesso de 30% para 66%. Como a rota ótima foi alcançada em grande parte das execuções, adotamos a inversão como padrão.

4.3. Evolução do Algoritmo Genético

Inicialmente, nossa modelagem do AG usava rotas iniciais aleatórias, função de fitness $1/(1 + f(\pi))$ e mutação por troca simples.

Primeiro, incluímos a lógica do vizinho mais próximo para uma das rotas iniciais, mantendo as outras 49 aleatórias. Esperávamos que isso favoreceria a reprodução. O desvio mediano do melhor indivíduo inicial caiu de 92,93% para 21,37%, mas o desvio final subiu de 6,06% para 12,57% e o sucesso caiu de 26% para 10%. A Figura A.4 mostra que houve uma perda muito rápida de diversidade, por isso rejeitamos essa mudança.

Em seguida, tentamos preservar apenas o melhor pai e completar a população com os 49 melhores filhos, descartando quase todos os pais sempre. Esperávamos que isso fosse melhorar a

diversidade. A Figura A.5 mostra que isso foi atingido: a mediana final foi de 50 ciclos distintos, contra 1 na política original. Porém, o desvio mediano subiu de 6,06% para 37,72%, sem nunca atingir o ótimo. Por isso, descartamos a mudança, entendendo que a maior diversidade não se refletiu em um maior desempenho nesse experimento.

Mantendo a população aleatória e a sobrevivência original, substituímos a mutação por troca pela inversão, esperando aproveitar a geometria das rotas, como na TS. A Figura A.6 mostra a vantagem crescente da inversão. O desvio mediano caiu de 6,06% para 0,37% e o sucesso aumentou de 26% para 40%. A inversão melhorou a mediana nas cinco instâncias e foi adotada.

Por fim, com a inversão fixada, testamos o fitness por ranking linear. A hipótese era que ele daria uma vantagem mais clara às melhores rotas durante a seleção. Entendíamos que no fitness inverso do custo, rotas com custos próximos recebiam aptidões muito parecidas e, por isso, tinham chances quase iguais na roleta. O desvio mediano caiu de 0,37% para zero e o sucesso aumentou de 40% para 58%. A Figura A.7 mostra a distribuição final. Pelas melhoras demonstradas, adotamos a mudança.

5. Divisão de Trabalho e Carga Horária

O desenvolvimento deste projeto ocorreu de forma integralmente colaborativa. Ambos dedicaram 10 horas ao trabalho, totalizando 20 horas de esforço conjunto. Para garantir o alinhamento técnico, as tarefas foram divididas e desenvolvidas em paridade nas seguintes frentes:

- **Fundamentação e Modelagem (3 horas por pessoa):** Revisão teórica dos algoritmos, definição da modelagem do problema e discussões
- **Desenvolvimento Computacional (3 horas por pessoa):** Modelagem e implementação da Têmpera Simulada e do Algoritmo Genético.
- **Avaliação e Documentação (4 horas por pessoa):** Discussões sobre a metodologia e formatação dos testes, implementação dos scripts e redação do artigo

6. Comparação

Para comparar as versões finais dos algoritmos, não conseguimos mais usar os testes usados até então, pois para ambos já estava muito fácil. Portanto tivemos que recorrer à comparação com o algoritmo guloso do vizinho mais próximo. Usamos dez instâncias de 40 cidades, com 30 execuções de TS e de AG por instância e 50.000 avaliações por execução, ajustando o resfriamento da TS ao novo orçamento. Na Figura A.8, seu custo representa 100%, ou seja, valores menores indicam melhoria sobre o guloso, não distância do ótimo, que não foi calculado nessa etapa.

A TS foi melhor nesta bateria, com redução mediana de 17,60% contra 15,50% do AG e metade do tempo de busca. Uma possível explicação é que a TS já parte de uma rota gulosa, enquanto o AG precisa evoluir uma população aleatória. Não conseguimos isolar esse possível efeito da inicialização.

Num geral, o guloso é rápido, mas não revê escolhas que podem encarecer a rota. Já TS e AG permitem revisá-las, mas com maior custo computacional e sem garantir o ótimo. Uma busca cega que enumere todas as rotas encontra o ótimo, mas enfrenta crescimento fatorial. Entre as buscas informadas, A* pode garantir o ótimo com uma heurística que não superestime o custo restante e tratamento adequado de estados repetidos, porém pode exigir muito tempo e memória [Russell and Norvig 2022].

Referências

Russell, S. J. and Norvig, P. (2022). *Artificial Intelligence: A Modern Approach*. Pearson Education Limited, 4 edition. Global Edition.

A. Gráficos

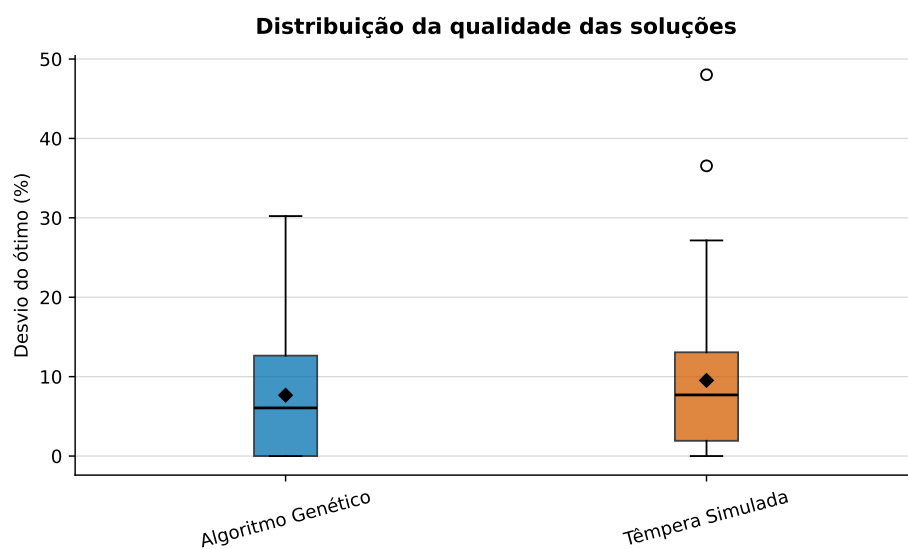


Figura A.1. Distribuição do desvio percentual do ótimo nas configurações iniciais. Cada algoritmo possui 50 execuções em cinco instâncias.

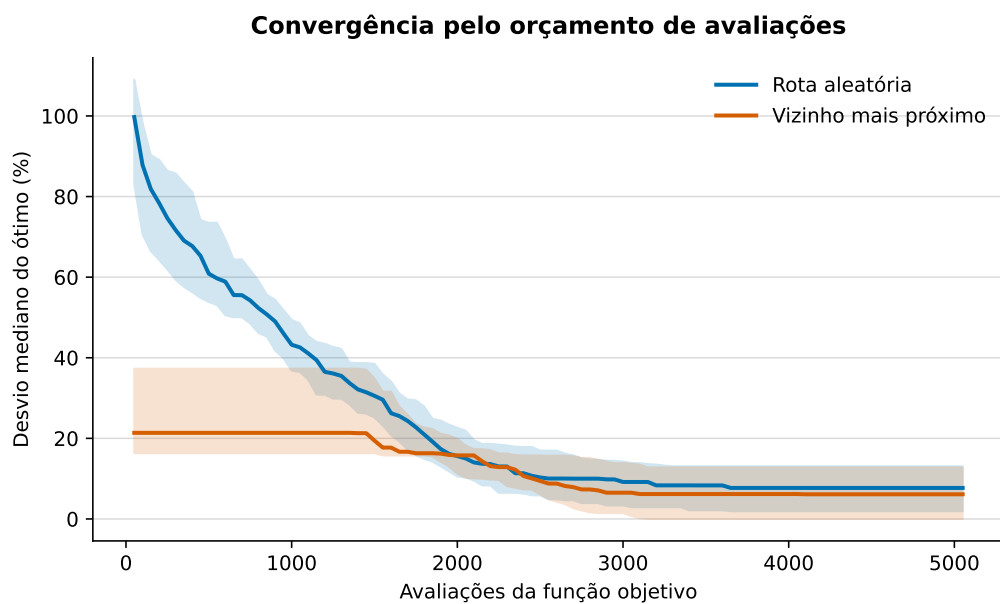


Figura A.2. Convergência da Têmpera Simulada com rota inicial aleatória e construída pelo vizinho mais próximo. A linha representa o desvio mediano e a faixa contém os 50% centrais dos resultados.

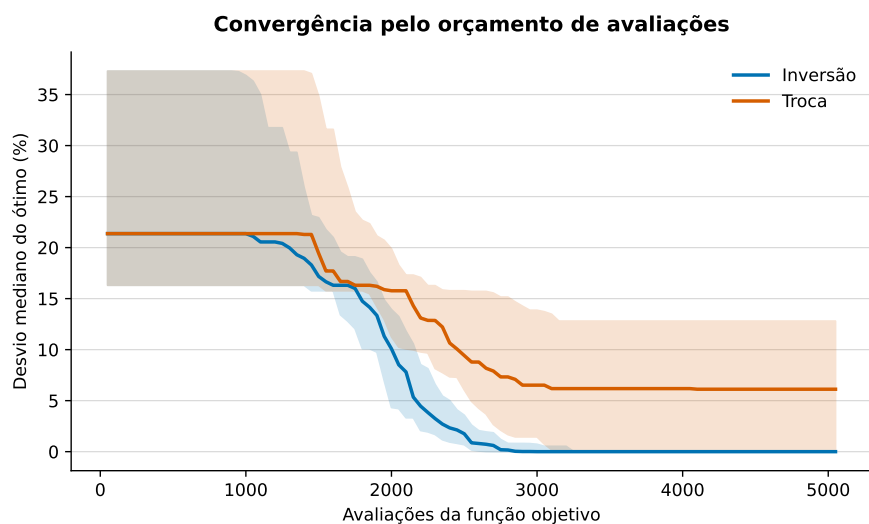


Figura A.3. Convergência da Têmpera Simulada com vizinhança por inversão de intervalo e por troca de duas cidades. As linhas representam as medianas e as faixas contém os 50% centrais dos resultados.

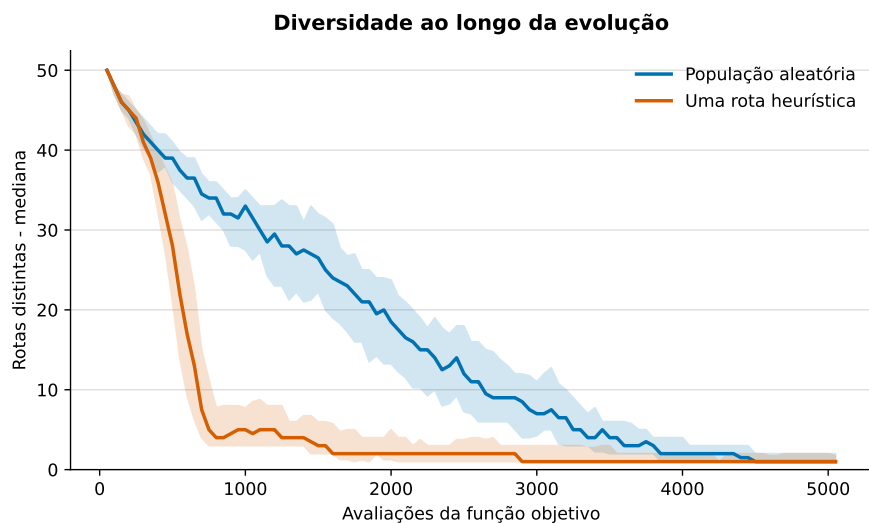


Figura A.4. Diversidade do AG com população totalmente aleatória e com uma rota inicial construída pelo vizinho mais próximo. As linhas indicam a mediana de ciclos distintos e as faixas os 50% centrais, em 50 execuções por configuração. Rotações e sentidos inversos contam como o mesmo ciclo.

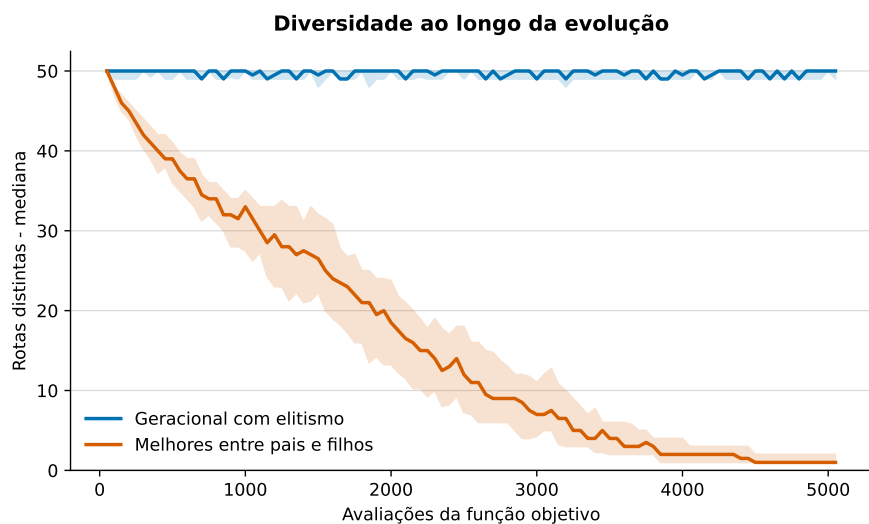


Figura A.5. Diversidade do AG nas duas políticas de sobrevivência. As linhas indicam a mediana de ciclos distintos e as faixas os 50% centrais, em 50 execuções por configuração. Rotações e sentidos inversos contam como o mesmo ciclo. Maior diversidade não implicou menor custo neste experimento.

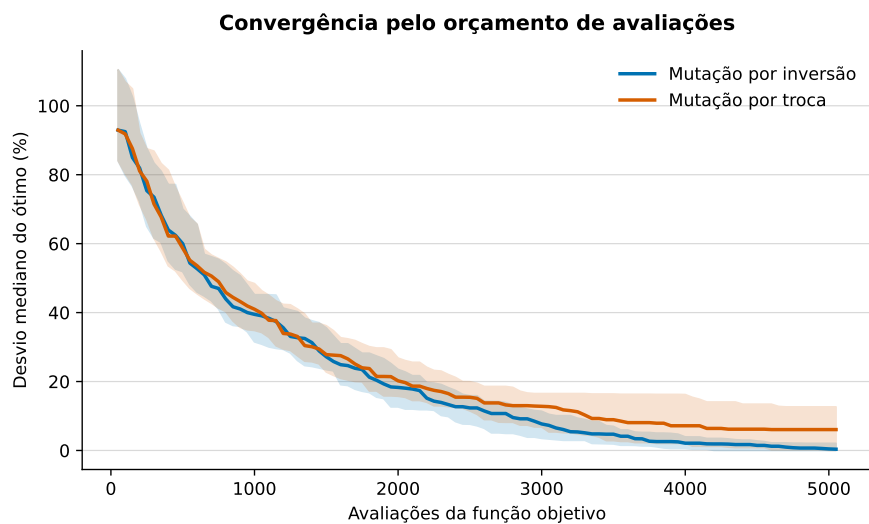


Figura A.6. Convergência do AG com mutação por troca e por inversão, ambas com fitness inverso do custo. As linhas indicam o desvio mediano e as faixas os 50% centrais, em 50 execuções por configuração.

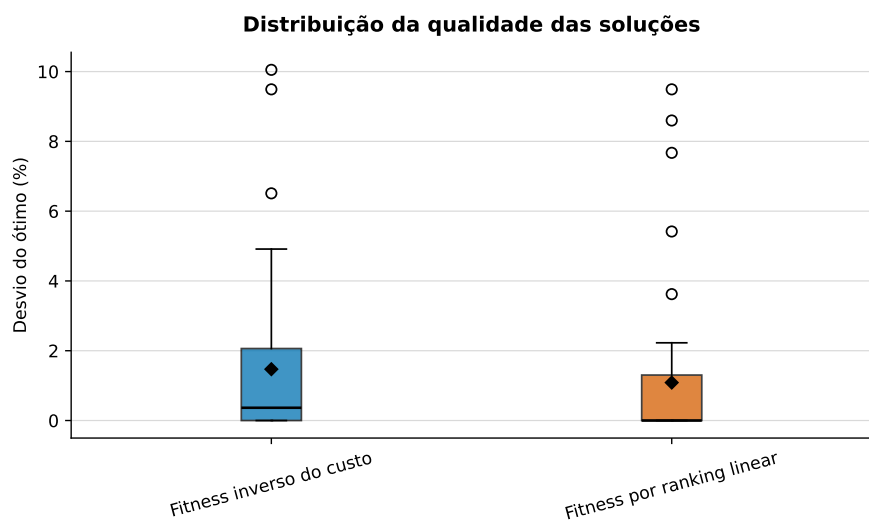


Figura A.7. Desvios finais do AG com fitness inverso do custo e por ranking, ambos com mutação por inversão. Cada configuração reúne 50 execuções. A linha indica a mediana, a caixa os 50% centrais, o losango a média e os círculos os valores atípicos.

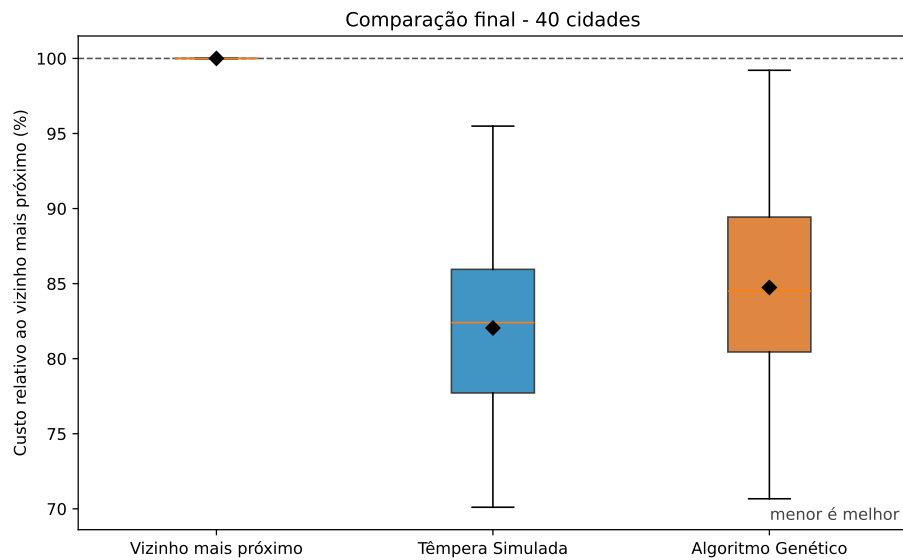


Figura A.8. Comparação final em dez instâncias de 40 cidades. Cada custo foi dividido pelo custo guloso da mesma instância, que representa 100%. TS e AG possuem 300 execuções cada, com 50.000 avaliações por execução; o guloso possui dez. A linha interna indica a mediana, a caixa os 50% centrais e o losango a média. O gráfico não mede distância do ótimo.